GOLEM Language Specification

Grid-Oriented Logic for Entity Manipulation

Akash Gupta	Kaushal Bule	Siddhesh Umarjee
23110020	23110160	23110347

1 Overview

GOLEM (Grid-Oriented Logic for Entity Manipulation) is a domain-specific programming language designed for specifying the behavior of autonomous agents operating within a discrete two-dimensional grid world. The language focuses on spatial reasoning, deterministic execution, and concise behavioral descriptions.

A GOLEM program defines:

- The geometry of the environment
- Static objects such as obstacles
- Behavioral templates called blueprints
- Initialization of the simulation through agent spawning

Execution produces a final world state describing positions, orientations, and inventories of all agents.

2 Fundamental Concepts

2.1 Grid World

- Finite rectangular grid
- Integer coordinate system
- Origin located at the top-left corner $(0, 0)$
- X increases eastward
- Y increases southward

Each grid cell may contain empty space, an obstacle, items, or a golem.

2.2 Golems

Golems are autonomous agents that:

- Occupy exactly one grid cell
- Maintain orientation (north, south, east, west)
- Execute instructions sequentially
- Maintain an inventory of items

2.3 Blueprints

Blueprints are reusable behavior templates. Multiple golems may execute the same blueprint definition.

3 Lexical Elements

3.1 Keywords

grid Declares grid dimensions and establishes coordinate bounds.

obstacle Places an immovable barrier that blocks movement.

blueprint Defines a reusable behavioural template for golems.

construct Begins simulation initialisation where agents are spawned.

spawn Instantiates a golem using a blueprint.

at Specifies spawn coordinates.

as Specifies aliases to blueprint objects.

go Moves the agent forward relative to orientation.

turn Changes orientation to a cardinal direction.

scan Inspects an adjacent grid cell without altering state.

pick Transfers an item into inventory.

drop Places an inventory item into the current cell.

if Conditional execution construct.

repeat Executes statements a fixed number of times.

north, south, east, west
Directional constants.

true, false Boolean literals.

3.2 Operators

== Equality comparison.

!= Inequality comparison.

? Marks a directional operand as a scan query.

3.3 Directional Symbols

>> East scan shorthand

<< West scan shorthand

^^ North scan shorthand

vv South scan shorthand

3.4 Delimiters

```
{ } ( ) ; ,
```

3.5 Identifiers

```
[a-z][a-zA-Z0-9_]*
```

3.6 Literals

- Integer values
- String literals for optional logging

4 Program Structure

A GOLEM program is composed of three ordered sections:

1. Grid Declaration
2. Blueprint Definitions
3. Simulation Construction

5 Language Constructs

5.1 Grid Declaration

```
grid(width, height) {
    obstacle(x, y);
}
```

5.2 Blueprint Definition

```
blueprint name {
    statements
}
```

5.3 Simulation Construction

```
construct {
    spawn blueprintName at (x,y);
}
```

6 Statements

6.1 Movement

```
go n;
```

6.2 Rotation

```
turn north;
turn south;
turn east;
turn west;
```

6.3 Environment Scan

```
scan(direction?) == obstacle
scan(direction?) == empty
```

6.4 Item Interaction

```
pick item;
drop item;
```

6.5 Conditional Execution

```
if(condition) {
    statements
}
```

6.6 Repetition

```
repeat k {
    statements
}
```

7 Execution Model

1. Grid initialized
2. Obstacles placed
3. Agents spawned
4. Each agent executes blueprint sequentially
5. Execution terminates when instructions complete

8 Semantic Constraints

- Movement outside grid is prohibited
- Collision with obstacles prevented
- Invalid spawn locations rejected
- Dropping non-existent items rejected
- Scan directions must be valid

9 Output Model

Execution produces a world-state report containing:

- Grid dimensions
- Obstacle coordinates
- For each golem:
  - Position
  - Orientation
  - Inventory contents

10 Example Programs

10.1 Simple Movement

```
grid(5,5) {}

blueprint walker {
    go 3;
    turn south;
    go 1;
}

construct {
    spawn walker as W1 at (0,0);
}
```

10.2 Obstacle Avoidance

```
grid(6,6) {
    obstacle(2,0);
}

blueprint avoider {
    repeat 3 {
        if(scan(east?) == obstacle) {
            turn south;
        }
        go 1;
    }
}

construct {
    spawn avoider at (0,0);
}
```

10.3 Item Transfer

```
grid(4,4) {}

blueprint collector {
    pick gold;
    go 1;
    drop gold;
}

construct {
    spawn collector at (1,1);
}
```

11 Conclusion

GOLEM provides a compact and expressive language for modeling autonomous agent behavior within structured environments. Its constrained syntax and deterministic semantics enable clear reasoning about spatial actions while remaining accessible for experimentation and demonstration.